CAN Signal Mapping Guide

1. Introduction

This guide explains how to map signals from the DBC (CAN database) file to the LVGL display and how to configure buttons to transmit CAN messages in the HMI Manager.

Key Files

File	Do you edit it?	Purpose
`app_hmi_manager.c`	**YES**	Implements the RX logic (CAN → Display) and TX logic (Buttons → CAN)
`can/car_can.h`	**NO (read only)**	Look up IDs, lengths, signals and structs (generated by cantools)
`can/car_can.c`	**NO**	Generated implementation (do not touch)
`uiux/widgets.c`	Rarely	Ready-made LVGL widgets

2. Data Flow

CAN Reception → Display (RX)

```
CAN bus
   |
FDCAN driver (kernel)
   |
SocketCAN network interface "can0"
(raw socket: socket(PF_CAN, SOCK_RAW, CAN_RAW))
   |
can_handler_read()     <- reads one struct can_frame
   |
process_can_frame()    <- YOU EDIT HERE
   |
     1. Identify the message ID
   2. Unpack:  car_can_*_unpack()
   3. Decode:  car_can_*_decode()
   4. Update the global variable
   |
widgets_set_speed()    <- updates LVGL
widgets_set_voltage()
   |
LCD display
```

Buttons → CAN Transmission (TX)

```
Button pressed (IRQ)
   |
button_irq_get_pressed()
   |
process_button_press() <- YOU EDIT HERE
   |
   1. Look up the g_button_map[] table
     2. Pack:  car_can_*_pack()
   3. Send:  can_handler_send()
   |
CAN bus
```

3. Understanding car_can.h

> **IMPORTANT:** `car_can.h` is generated automatically by cantools from the DBC. You do NOT edit this file, you only read it.

car_can.h Structure

Part 1: Message IDs

```
/* Frame ids. */
#define CAR_CAN_INVERTER_DEMANDERS_FRAME_ID (0xc09fefeu)
#define CAR_CAN_SYSTEM_VOLTAGES_FRAME_ID    (0xc06fe00u)
#define CAR_CAN_DRIVER_COMMANDS_FRAME_ID    (0xc03fe32u)
```

How to use: copy the exact name into the `if (id == ...)` comparison.

Part 2: ID Type (Standard vs. Extended)

```
#define CAR_CAN_SYSTEM_VOLTAGES_IS_EXTENDED (1) /* 1 = 29-bit ID */
```

Part 3: Message Structures

```
struct car_can_system_voltages_t
{
  uint16_t meas12_v;          /* 12 V rail voltage */
  uint8_t  meas5_v;           /* 5 V rail voltage  */
  uint16_t meas_dc_link;      /* DC link voltage   */
  uint16_t meas_batt_voltage; /* Battery voltage   */
};
```

Important information:

- Struct name: `car_can_system_voltages_t`
- Available signals: `meas12_v`, `meas5_v`, `meas_dc_link`, `meas_batt_voltage`
- Type of each field: `uint16_t`, `uint8_t`, etc.

Part 4: Encode/Decode Functions

```
/* Decode: raw value -> physical value */

double car_can_system_voltages_meas_batt_voltage_decode(
  uint16_t value);

/* Encode: physical value -> raw value */

uint16_t car_can_system_voltages_meas_batt_voltage_encode(
  double value);
```

When to use:

- RX: use `decode()` to convert the CAN value into a physical value.
- TX: use `encode()` to convert a physical value into the CAN value.

4. Reception: CAN → Display

Complete Step by Step

STEP 1: Find the message that carries the desired signal

Open `car_can.h` and search for keywords related to the signal.

Example: I want to show the vehicle speed on the display. Search for: `speed`, `velocity`, `kmh`.

Found (hypothetical):

```
struct car_can_vehicle_status_t
{
  uint16_t vehicle_speed; /* Vehicle speed      */
  uint8_t  motor_temp;    /* Motor temperature  */
  uint8_t  motor_state;   /* Motor state        */
};

#define CAR_CAN_VEHICLE_STATUS_FRAME_ID    (0x18ff1234u)
#define CAR_CAN_VEHICLE_STATUS_IS_EXTENDED (1)
```

STEP 2: Check the signal scaling

Look at the comments in `car_can.h`:

```
/**
 * Range: 0..65535 (0..655.35 km/h)
 * Scale: 0.01
 * Offset: 0
 */

uint16_t vehicle_speed;
```

Interpretation:

- Raw value on CAN: 0 to 65535
- Physical value: 0.00 to 655.35 km/h
- Conversion: `physical = raw × 0.01`

STEP 3: Edit app_hmi_manager.c

Locate the `process_can_frame()` function (line ~100-200) and add:

```
/* Vehicle speed */

if (id == CAR_CAN_VEHICLE_STATUS_FRAME_ID)
  {
    struct car_can_vehicle_status_t msg;

    ret = car_can_vehicle_status_unpack(&msg, frame->data,
                                        frame->can_dlc);
    if (ret == 0)
      {
        /* Decode speed (raw -> km/h) */

        double speed =
          car_can_vehicle_status_vehicle_speed_decode(
            msg.vehicle_speed);

        /* Update global variable */

        g_speed = (int)speed;

        /* Update display */

        widgets_set_speed(g_speed);
      }
  }
```

STEP 4: Build and test

```
make -j$(nproc)
make flash
```

Test from Linux (assuming `vehicle_speed` starts at bit 0, little-endian, in the DBC):

```
# Send vehicle_speed = 80 km/h
# 80 / 0.01 = 8000 = 0x1F40
# Little-endian at bytes 0-1: 40 1F
cansend can0 18FF1234#401F000000000000
```

Generic RX Template

```
if (id == CAR_CAN_[MESSAGE_NAME]_FRAME_ID)
  {
    struct car_can_[message_name]_t msg;

    ret = car_can_[message_name]_unpack(&msg, frame->data,
                                        frame->can_dlc);
    if (ret == 0)
      {
        /* Decode signal */

        double value =
          car_can_[message_name]_[signal_name]_decode(
            msg.[signal_name]);

        /* Update variable */

        g_[variable] = (int)value;

        /* Update widget */

        widgets_set_[widget](g_[variable]);
      }
  }
```

5. Transmission: Buttons → CAN

Button Architecture

The system uses a configuration table to map buttons to CAN messages.

Configuration Structure

```
struct button_config_s
{
  uint32_t msg_id;   /* CAN message ID                */
  bool     extended; /* Extended (29-bit) ID flag     */
  uint8_t  data[8];  /* Pre-packed CAN payload        */
  uint8_t  dlc;      /* Data Length Code (0-8)        */
};
```

Create the Mapping Table

At the top of `app_hmi_manager.c`, after the includes:

```
/* Button mapping table */

static const struct button_config_s g_button_map[] =
{
  /* BTN0: Hazard lights */

  {
    .msg_id   = CAR_CAN_BUTTON_COMMANDS_FRAME_ID,
    .extended = CAR_CAN_BUTTON_COMMANDS_IS_EXTENDED,
    .data     = {0x00, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00},
    .dlc      = 2
  },

  /* BTN1: Left turn signal */

  {
    .msg_id   = CAR_CAN_BUTTON_COMMANDS_FRAME_ID,
    .extended = CAR_CAN_BUTTON_COMMANDS_IS_EXTENDED,
    .data     = {0x01, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00},
    .dlc      = 2
  },

  /* BTN2: Right turn signal */

  {
    .msg_id   = CAR_CAN_BUTTON_COMMANDS_FRAME_ID,
    .extended = CAR_CAN_BUTTON_COMMANDS_IS_EXTENDED,
    .data     = {0x02, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00},
    .dlc      = 2
  },

  /* Add more buttons as needed */
};

#define NUM_BUTTONS (sizeof(g_button_map) / sizeof(g_button_map[0]))
```

Calculating the Byte Values

Method 1: Manual (fast)

If the message is simple, compute it directly.

Example: the ButtonCommands message has 2 bytes:

```
Byte 0: button_id    (0-255)
Byte 1: button_state (0 = released, 1 = pressed)
```

For BTN3 (horn):

```
.data = {0x03, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}
          ^^^^  ^^^^
           |     +-- button_state = 1 (pressed)
           +-------- button_id    = 3 (horn)
```

Method 2: Use the DBC functions (recommended)

```
/* Helper for BTN3 */

static void pack_btn3(uint8_t *data, size_t size)
{
  struct car_can_button_commands_t msg;

  car_can_button_commands_init(&msg);
  msg.button_id    = 3; /* Horn */
  msg.button_state = 1; /* Pressed */
  car_can_button_commands_pack(data, &msg, size);
}
```

6. Complete Practical Example

Scenario

Implement:

1. The display shows the battery voltage (already in the current DBC).
2. Button 5 sends a reset command.

Part 1: Battery Voltage (RX)

Look up car_can.h

```
struct car_can_system_voltages_t
{
  uint16_t meas12_v;
  uint8_t  meas5_v;
  uint16_t meas_dc_link;
  uint16_t meas_batt_voltage; /* <- THIS ONE */
};

#define CAR_CAN_SYSTEM_VOLTAGES_FRAME_ID (0xc06fe00u)
```

Check the Scaling

```
/**
 * Range: 0..65535 (0..655.35 V)
 * Scale: 0.01
 * Offset: 0
 */

uint16_t meas_batt_voltage;
```

Example: CAN = 4850 → physical = 4850 × 0.01 = 48.5 V

Implement

```
/* Battery voltage */

if (id == CAR_CAN_SYSTEM_VOLTAGES_FRAME_ID)
  {
    struct car_can_system_voltages_t msg;

    ret = car_can_system_voltages_unpack(&msg, frame->data,
                                         frame->can_dlc);
    if (ret == 0)
      {
        /* Decode (returns volts) */

        double voltage =
          car_can_system_voltages_meas_batt_voltage_decode(
            msg.meas_batt_voltage);

        /* Widget expects decivolts (0.1 V), multiply by 10 */

        g_voltage = (int)(voltage * 10.0);
        widgets_set_voltage(g_voltage);
      }
  }
```

Test

The bytes in the frame depend on the start bit of `meas_batt_voltage` in the DBC, which is not visible in `car_can.h`. The example below assumes start bit 48 (byte 6), little-endian:

```
# Send 48.5 V
# 48.5 / 0.01 = 4850 = 0x12F2
# Little-endian at bytes 6-7: F2 12
cansend can0 0C06FE00#000000000000F212
```

Part 2: Reset Button (TX)

Add to the Table

```
/* BTN5: Reset command */

{
  .msg_id   = CAR_CAN_DEBUG_DEMANDS_FRAME_ID,
  .extended = CAR_CAN_DEBUG_DEMANDS_IS_EXTENDED,
  .data     = {0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00},
  .dlc      = 1
},
```

7. Troubleshooting

Display does not update

Diagnosis:

1. Enable the CAN trace: `hmi_ctl can --trace show`
2. If the message shows in the trace but the display does not change → error in `process_can_frame()`
3. If it does not show in the trace → problem in the physical CAN layer

Solution:

- Check that the ID is correct
- Add a temporary `printf()` for debugging
- Check that the `decode()` function is being called

Wrong value on the display

Common cause: forgot to decode or applied the wrong scaling.

```
/* WRONG */

g_voltage = (int)msg.voltage_v; /* Uses the raw value */

/* RIGHT */

double v = car_can_system_voltages_meas_batt_voltage_decode(
             msg.voltage_v);
g_voltage = (int)(v * 10.0);
```

Button does not send CAN

Diagnosis:

1. Add `printf("Button %d ", button);` in `process_button_press()`
2. Check that `NUM_BUTTONS` is correct
3. Check that the table has an entry for that button

8. Checklist

For RX (CAN → Display)

☐ Looked up `car_can.h` and found the correct message/signal
☐ Copied the exact message ID
☐ Added `if (id == ...)` in `process_can_frame()`
☐ Called `*_unpack()` correctly
☐ Called `*_decode()` to get the physical value
☐ Updated the widget with `widgets_set_*()`
☐ Tested with `cansend` on Linux
☐ Display updated correctly

For TX (Buttons → CAN)

☐ Created an entry in the `g_button_map[]` table
☐ Set `msg_id`, `extended`, `data`, `dlc`
☐ Computed the bytes correctly (manually or with `_pack()`)
☐ Built without errors
☐ Tested by pressing the physical button
☐ Message showed up in the CAN trace
☐ Target ECU (Electronic Control Unit) received it correctly

9. Review Notes

9.1 Corrections applied in this version

1. **SocketCAN vs. /dev/can0 (section 2):** SocketCAN is a network interface (`can0`) opened through a socket. `/dev/can0` is the NuttX CAN character driver, which uses `struct can_msg_s` instead of `struct can_frame`. Since the code uses `frame->can_dlc`, it is the SocketCAN path.
2. **RPM example (section 4):** the previous version decoded a motor RPM (Revolutions Per Minute) signal and passed it to `widgets_set_speed()`. RPM is not vehicle speed, so the example now uses a vehicle speed signal. To display RPM, create a dedicated widget (e.g. `widgets_set_rpm()`) and variable.
3. **Battery voltage test (section 6):** the payload position depends on the signal start bit in the DBC, which `car_can.h` does not show. The assumption (start bit 48, little-endian) is now stated explicitly.

9.2 Suggested corrections (not applied)

1. **Extended ID flags:** in SocketCAN, `can_id` carries `CAN_EFF_FLAG` (bit 31) for 29-bit frames, so `frame->can_id` never equals the `*_FRAME_ID` value directly. Unless `process_can_frame()` already does it, strip the flags first:

```
uint32_t id;

if ((frame->can_id & CAN_EFF_FLAG) != 0)
  {
    id = frame->can_id & CAN_EFF_MASK; /* 29-bit ID */
  }
else
  {
    id = frame->can_id & CAN_SFF_MASK; /* 11-bit ID */
  }
```

On TX, `can_handler_send()` must OR `CAN_EFF_FLAG` into `can_id` when `extended` is true.

2. **Hard-coded DLC (Data Length Code):** `.dlc = 2` and `.dlc = 1` must match the message length in the DBC. Use the generated `CAR_CAN_*_LENGTH` macros instead; a receiver that checks the DLC (or a cantools `*_unpack()`) rejects a shorter frame.
3. **Non-existent field (section 7):** `msg.voltage_v` does not exist in `car_can_system_voltages_t`; it should be `msg.meas_batt_voltage`.
4. **Manual byte calculation (section 5, Method 1):** only valid if the signals are byte-aligned, with scale 1 and offset 0.
5. **Method 2 helper:** `*_pack()` runs at run time, so it cannot fill a `static const` initializer. Call it in `process_button_press()` or at startup to fill a non-const table.
6. **Table index:** BTN5 must be at index 5 of `g_button_map[]`; missing BTN3/BTN4 entries shift it to the wrong button.
7. **Payload generation:** to avoid start-bit assumptions, let cantools compute the test payload (message and signal names must match the DBC):

```
python3 - <<'EOF'
import cantools

db  = cantools.database.load_file('car.dbc')
msg = db.get_message_by_name('SystemVoltages')
raw = msg.encode({'Meas12V': 0, 'Meas5V': 0, 'MeasDcLink': 0,
                  'MeasBattVoltage': 48.5})
print('cansend can0 %08X#%s' % (msg.frame_id, raw.hex().upper()))
EOF
```

8. **make flash:** only works if the board `Make.defs` defines the `FLASH` macro; otherwise flash `nuttx.bin` with OpenOCD or STM32CubeProgrammer.
9. **Line reference:** "line ~100-200" for `process_can_frame()` goes stale as the file changes; reference the function name only.
10. **LVGL thread safety:** LVGL is not thread-safe. Call `widgets_set_*()` only from the thread that runs `lv_timer_handler()`, or protect it with the same lock.
11. **Debugging:** prefer `syslog()` over `printf()` in NuttX, and use `candump can0` on the Linux side to confirm the frame is on the bus. If it is not there, check bitrate, termination (120 Ω at each end) and wiring.